

School of Electronic Engineering
and Computer Science

Interim Report

Programme of study:
Computer science

Project Title:
**Incorporating critic's
uncertainty into
proximal policy
optimization for
improved decision
making**

Supervisor:
Professor Paulo Rauber

Student Name:
Milkesa Haji Nebo

Final Year
Undergraduate Project 2024/25



Date: Add submission date

Abstract

Write your abstract here.

Contents

Chapter 1:	Introduction	3
1.1	Background	3
1.2 Problem Statement		3
1.3 Aim		3
1.4 Objectives		3
1.5 Research Questions		3
1.6 Report Structure.....		3
Chapter 2: Literature Review.....		3
2.1 Second Level Headings		3
Chapter 3: Experimentation.....		3
Chapter 4: Presentation		3
4.1 Source Code		3
Chapter 5: Evaluation.....		3
Chapter 6: Conclusion.....		3
References		3
Appendix A - Example		3
Additional Appendices (as needed).....		3

Chapter 1: Introduction

1.1 Background

Autonomous driving is a field where safety is the priority, and where the driving agent must make continuous decisions in uncertain and dynamic traffic conditions. Unlike controlled environments, such as an autonomous robotic arm in a factory environment, traffic scenarios involve surrounding cars whose positions and speeds vary over time. This makes decision making challenging because an autonomous vehicle has to consider both itself and other vehicles to ensure safe driving while responding to unpredictable situations.

Reinforcement Learning (RL) is well suited to this type of problem because it trains an agent to make sequential decisions through interactions with the environment. In this field, an RL agent can learn behaviours such as overtaking, lane keeping and collision avoidance through rewards for desirable actions and penalties for undesirable actions. Proximal Policy Optimisation (PPO) is commonly used in RL because it provides stability and is practical to implement.

However, the performance of the agent depends heavily on how the driving state is represented. In a highway environment the driving state includes the positions and speeds of the agent and nearby cars. A standard multilayer perceptron (MLP) processes these observations as a flattened vector, which makes the policy sensitive to vehicle ordering. Permutation-invariant architectures may provide a more suitable representation by treating surrounding vehicles as sets rather than an ordered list.

Another challenge is that standard PPO does not explicitly account for uncertainty. This means the policy may select actions without considering how uncertain the model is, which could lead to collisions in unfamiliar situations where the agent has limited experience. This motivates the use of Monte Carlo dropout as a lightweight method for estimating uncertainty.

1.2 Problem Statement

A key challenge in using reinforcement learning for autonomous driving is that the agent has to deal with traffic situations that it may not have seen during training. In unfamiliar situations, overconfident decisions can result in collisions or unstable driving behaviour. Therefore, an autonomous driving policy should also take into account the uncertainty of its decisions.

PPO uses an actor-critic architecture in which the critic computes a value estimate of how 'good' a state is. This estimate does not show how confident the critic network is in its prediction. If the critic produces an unreliable value estimate in a particular traffic scenario, the agent may receive an inaccurate learning signal, which can lead to unsafe driving behaviour during training.

This creates a problem because the agent may continue learning from uncertain value estimates without a mechanism for discouraging uncertain behaviour. The project addresses this issue by investigating whether penalising uncertainty during training improves performance and, most importantly, safety compared with the baseline.

1.3 Aim

This project investigates whether incorporating epistemic uncertainty into PPO can improve the performance and safety of an autonomous driving agent in a simulated highway environment. A baseline PPO agent is first implemented and evaluated using different policy network backbones, including permutation-invariant architectures. The selected backbone is then extended with Monte Carlo dropout in the critic network to compute an uncertainty approximation. The uncertainty approximation is used in training as a reward penalty, and this agent is then compared against the baseline using average return, episode length and collision rate.

1.4 Objectives

- To implement a baseline Proximal Policy optimisation (PPO) agent in the highway-env environment to establish a performance benchmark.
- To compare different policy network backbones, including MLP, Deep Sets and self-attention, and then select the strongest architecture for the final uncertainty-aware PPO model.
- To modify the critic network by incorporating Monte Carlo dropout to obtain an estimate of epistemic uncertainty during value prediction.
- To develop an uncertainty-aware policy optimisation mechanism that integrates uncertainty estimates into the PPO learning process.
- To train and evaluate the baseline and uncertainty-aware PPO agents under identical conditions.
- To analyse the impact of uncertainty estimation on performance and safety using metrics such as cumulative reward, collision rate and policy stability.

1.5 Research Questions

- How does the choice of policy network architecture affect PPO performance in an autonomous driving task?
- Does penalising the critic's uncertainty during PPO training improve safety compared with the baseline PPO agent?
- How does uncertainty penalisation affect the trade-off between episodic reward and collision rate?

1.6 Report Structure

Chapter 2 reviews reinforcement learning and autonomous driving, PPO, permutation-invariant architectures and uncertainty estimation via Monte Carlo dropout. Chapter 3 describes the experimental setup, highway-env configuration, PPO implementation, backbone comparison and uncertainty-aware extension implementation. The final chapter concludes the report by summarising the findings, limitations and future work.

Chapter 2: Literature Review

2.1 Literature review overview

This chapter reviews literature relevant to the project. The review focuses on three main areas: reinforcement learning for autonomous driving, with a focus on PPO; uncertainty estimation methods, with a focus on Monte Carlo dropout; and permutation-invariant architectures for multi-object observations, such as Deep Sets and self-attention. Together, these areas provide the theoretical foundation for the two main stages of this project: backbone selection and the uncertainty-aware PPO extension.

2.2 Reinforcement Learning for autonomous driving

Reinforcement Learning (RL), a subfield of Artificial Intelligence (AI), focuses on training agents to make decisions by interacting with their environment to maximise cumulative rewards [1]. RL is especially well suited to sequential decision-making problems because it learns policies that optimise long-term cumulative reward in an environment where each action influences future states and future decisions. For example, in autonomous driving, a lane change made by the ego vehicle at one moment can alter future traffic interactions.

Autonomous driving is well suited to an RL formulation because it is a sequential decision-making problem in a dynamic and uncertain environment, where the agent must continually adapt to changing traffic conditions while balancing safety and efficiency. This makes autonomous driving a good domain for investigating whether an RL agent can learn effective behaviour through repeated interactions with its environment.

Although autonomous driving is suited to RL, there are still several challenges. The ego vehicle must operate in the presence of multiple nearby vehicles, meaning that its decision depends on both its own state and the state of other vehicles. This creates a complex multi-agent scenario that is highly unpredictable. Autonomous driving is also safety-critical, as poor decisions may result in collisions or other unsafe events. These factors make autonomous driving a demanding field for RL, requiring the agent to learn policies that are not only efficient but also safe.

Because autonomous driving is safety-critical, training directly in real-world environments is impractical due to high safety risks, high costs and limited controllability. Allowing an agent to train on a public road would be nonsensical, particularly during the early stages where unsafe behaviour is almost guaranteed. Simulation therefore plays a crucial role, as it enables large amounts of experience to be generated without the safety risks and high costs of real-world testing. This makes simulation environments such as highway-env [2] especially important for developing autonomous vehicles using an RL approach.

Among reinforcement learning algorithms, Proximal Policy Optimisation (PPO) is suitable for the autonomous driving experiments in this project due to its relative simplicity, stability and strong empirical performance across benchmark tasks [3]. However, the use of PPO does not resolve important challenges such as uncertainty in unfamiliar states or the ability to make consistently safe decisions. These limitations motivate the two main parts of this project: improving the policy representation through backbone selection and extending PPO with uncertainty estimation in the critic network.

2.3 Proximal Policy Optimisation (PPO)

Proximal Policy Optimisation (PPO) is a policy-gradient reinforcement learning algorithm that improves training stability by restricting excessively large policy updates through a clipping mechanism between iterations, while still allowing efficient optimisation through repeated minibatch updates. It is split into two parts: the actor network and the critic network. The actor network selects actions, and the critic estimates the value of states. PPO has become widely used as a baseline for continuous-control reinforcement learning tasks.

Schulman et al. (2017) introduced PPO as a policy-gradient algorithm designed to improve training stability by limiting large policy updates. This has made PPO a widely used baseline because it balances implementation simplicity with strong empirical performance. In contrast with earlier reinforcement learning approaches such as Q-learning, PPO provides a more practical optimisation procedure that allows multiple minibatch updates while reducing the risk of destructive policy changes. This makes it easier to train reliably across a range of reinforcement learning tasks.

PPO is well suited to this project because it can learn sequential decision-making behaviour in a dynamic highway environment while supporting continuous control. In this project, the agent operates in a continuous action space, making decisions about the direction and speed of the vehicle, for which smooth policy outputs are desirable. PPO is also well suited to simulation-based training, as the simulation repeatedly provides new interaction data from the current policy, which is needed for PPO to be practical.

Despite these advantages, PPO does not resolve several important challenges in autonomous driving. In particular, it does not explicitly use uncertainty, meaning that the agent has no direct means of knowing how confident it is in unfamiliar and difficult states. Safe behaviour should also be encouraged through the reward system. Furthermore, PPO performance depends strongly on the quality of the observation representation and policy architecture. These limitations motivate the two main directions tackled in this project: comparing alternative policy backbones and providing an uncertainty-aware extension to the PPO algorithm.

2.4 Permutation Invariance

In autonomous driving tasks, the agent observes multiple surrounding vehicles, each with features such as velocity and position. When using a standard multi-layer perceptron (MLP), the model processes the observation as a flattened input vector and is therefore sensitive to the ordering of the vehicles. As a result, two similar traffic situations may produce different policy outputs if the observed vehicles are arranged in a different order. This motivates the use of permutation-invariant architectures for this reinforcement learning task.

Permutation invariance refers to the property that the output of a model should remain unchanged when the order of elements in a set is altered (Zaheer et al., 2017). In the context of autonomous driving, this is important because reordering the surrounding vehicles in the observation should not change the underlying meaning of the traffic situation, and therefore should not change the policy output. Deep Sets is a permutation-invariant neural network architecture proposed for learning over set-structured inputs while preserving this property. In a Deep Sets model, each element, in this case each vehicle, is processed individually through a shared encoder, and the resulting embeddings are then combined using a symmetric aggregation function, such as mean pooling or max pooling. As these functions are order-independent, the resulting

representation is permutation invariant, making Deep Sets a well suited choice for multi-vehicle reinforcement learning tasks.

Although symmetric aggregation results in a permutation-invariant representation, the choice of function affects the type of information captured in the final set representation. Mean pooling provides a summary of the overall traffic context by averaging features across observed vehicles. However, this averaging process may dilute the importance of some vehicles, making it harder for the agent to respond to a nearby slow or hazardous vehicle, which may increase collision risk. In contrast, max pooling preserves the strongest feature responses across the set, making it better suited to retaining dominant signals. For traffic-based decision making, a combination of both mean and max pooling may provide a richer final representation by capturing both a summary of the overall traffic context and the most relevant vehicle features.

Self-attention is a more flexible architecture than Deep Sets for processing traffic observations. Self-attention allows a model to represent each element of an input sequence or set by considering its relationship to other elements, enabling the model to assign different levels of importance to different inputs (Vaswani et al., 2017). Rather than summarising vehicle embeddings using aggregation functions, self-attention allows the model to assign different importance to different vehicles while also capturing relationships between them. This is significant in autonomous driving because the importance of a surrounding vehicle depends not only on its state but also on its relationship with the ego vehicle and other nearby vehicles. As a result, self-attention can, in principle, produce a richer and more context-sensitive traffic representation compared with a Deep Sets model. However, this greater expressiveness comes at the cost of increased model complexity, which may make training more difficult and require more data or tuning to achieve strong results.

2.5 Uncertainty in reinforcement learning

Uncertainty is an important consideration in reinforcement learning because agents are often required to make decisions where future outcomes are not predictable. When uncertainty is not accounted for, an agent may act unreliably in unfamiliar or complex situations, reducing both robustness and decision accuracy. This is important in autonomous driving, where traffic conditions change continuously and the behaviour of nearby vehicles can be unpredictable at times. In such tasks where safety is a priority, understanding when the model is uncertain may help identify situations in which more cautious behaviour is needed.

Uncertainty is commonly divided into epistemic and aleatoric uncertainty. Epistemic uncertainty arises from uncertainty in the model and can be reduced with additional data, whereas aleatoric uncertainty reflects noise inherent in the observations and cannot be removed simply by collecting more data (Kendall and Gal, 2017). For reinforcement learning, epistemic uncertainty is important as it may highlight situations in which the model lacks confidence due to limited experience. This is especially relevant when the agent encounters rare traffic patterns or complex interactions that were not covered well in training.

Estimating uncertainty in reinforcement learning can be challenging, especially when the model must operate over high-dimensional state spaces. A practical and widely used uncertainty approximation is Monte Carlo dropout. Monte Carlo dropout estimates uncertainty by keeping dropout active at inference time and performing multiple stochastic forward passes, with the spread of predictions used as an approximation of model uncertainty (Gal and Ghahramani, 2016). This makes Monte Carlo dropout a good

choice for uncertainty estimation, as it provides a relatively lightweight approximation compared with ensembles without requiring a full Bayesian neural network.

In autonomous driving, uncertainty approximation is valuable because it can help identify states in which the model is less reliable or has limited experience. In principle, this could support more cautious or better-informed decision making when the agent encounters unfamiliar traffic situations. However, PPO does not estimate uncertainty, which means that the policy operates without a direct measure of model confidence. This motivates the use of an uncertainty-aware extension to the standard PPO algorithm, in which uncertainty approximation is used to help the agent make better-informed decisions.

2.6 Related work and Research Gap

Reinforcement learning has been widely explored for autonomous driving, including tasks such as overtaking, lane keeping and collision avoidance in simulated traffic environments (Leurent and Mercat, 2019). Much of this work has been conducted in simulation, such as highway-env, where agents can be trained while avoiding the safety risks and costs of real-world testing (Farama Foundation, n.d.). Prior studies have shown that reinforcement learning can produce effective driving behaviour in controlled environments, indicating its potential for sequential traffic decision making. However, the literature also shows that achieving a robust and reliable agent remains challenging, especially in an unpredictable multi-vehicle environment where safety is critical (Gu et al., 2023).

Zaheer et al. (2017) showed that Deep Sets can learn from set-structured inputs while remaining invariant to the ordering of elements. This is relevant to autonomous driving because surrounding vehicles can be treated as a set rather than a fixed ordered vector. Vaswani et al. (2017) introduced self-attention, which allows a model to assign different importance to different input elements, such as vehicles in this project, and to capture relationships between them. However, while attention can provide a more informative representation, it also increases model complexity and may require more training data or tuning. This creates a practical question for this project: whether simpler permutation-invariant backbones, such as Deep Sets, are safer and more efficient than self-attention in autonomous driving.

Clements et al. (2019) showed that uncertainty estimation can be useful in reinforcement learning because it can identify states where the agent has limited confidence. Similarly, Zhao et al. (2023) explored uncertainty-aware reinforcement learning in safety-critical settings, highlighting that uncertainty can support safer decision making. Gal and Ghahramani (2016) proposed Monte Carlo dropout as a practical and lightweight approximation for model uncertainty, making it suitable for neural-network-based reinforcement learning systems. However, much of this work focuses on estimating uncertainty rather than evaluating how uncertainty estimates should be incorporated into the training process of an RL agent for autonomous driving. This leaves a gap for investigating whether critic uncertainty can be used to improve safety in a highway environment.

Overall, the literature in this field shows that reinforcement learning has strong potential in autonomous driving, while also highlighting the importance of both state representation and uncertainty-aware decision making. However, there remains value in a study that examines these two issues within a single experimental pipeline. In particular, there appears to be limited work that first compares different policy backbones and then evaluates uncertainty-aware extensions built on top of the selected policy. This project addresses that gap by comparing standard MLP, two Deep Sets models and self-

attention policy backbones within the environment, and then evaluating whether an uncertainty-aware extension can improve overall safety.

2.7 Comparison of Existing Literature

Study	Method	Relevance	Limitation
Schulman et al.	PPO	Core RL algorithm	Does not address uncertainty
Gal and Ghahramani	MC dropout	Epistemic uncertainty	Not specific to RL driving
Leurent and Mercat	Highway driving RL	Relevant environment/task	Not focused on uncertainty penalty
Clements et al.	Uncertainty in deep RL	Supports safety motivation	

Chapter 3: Experimentation

3.1 Experimental Overview

This chapter describes the experimental procedure used to evaluate different policy network backbones and the proposed uncertainty-aware extension introduced earlier. The first stage of experimentation focused on selecting the most suitable backbone architecture for the policy network, which was then used in both the baseline PPO model and the uncertainty-aware model. The second stage compared the performance of the baseline PPO model with the proposed uncertainty-aware model.

The experiments were conducted in the highway-env environment, where an autonomous vehicle drives on a three-lane highway. The agent is rewarded for avoiding collisions, maintaining a reasonable speed and making safe lane-selection decisions. The evaluation metrics used for comparison were average episodic return, average episodic length and collision rate. The models were trained and evaluated under controlled settings to ensure fair comparisons.

3.2 Highway-env setup

The environment used in this project was a customised version of the highway-v0 environment from the highway-env library. The RL agent drives on a multilane highway populated with other vehicles. The agent's objective is to reach a high speed while avoiding collisions with neighbouring vehicles. Driving on the right side of the road is also rewarded [2]. In the version used, there are three lanes, one ego vehicle controlled by the agent and surrounding traffic generated by the environment.

In the custom environment, traffic continuously spawns both behind and ahead of the ego vehicle, creating a dynamic driving scenario. The observation includes multiple nearby vehicles, with each vehicle represented by features such as presence, x-position, y-position relative to the agent, x-velocity and y-velocity. Urgency-related features were also appended to the flattened observation vector to provide extra information about nearby traffic risk. The action space is continuous rather than discrete, allowing smoother driving and lane-selection behaviour.

The reward configuration was designed to encourage safe and efficient driving. The agent was penalised for collisions, leaving the highway and lane changes to encourage safer driving and discourage unnecessary lane switches. The agent was also rewarded for maintaining higher speeds within the target range, which helps reduce rear-end collisions.

Overall, the setup was designed to test not just lane following, but also interaction with surrounding traffic. This makes it suitable for comparisons between different policy backbones and later evaluating the uncertainty-aware PPO model.

3.3 Policy network architectures

3.3.1 Basic MLP

The basic MLP architecture takes the flattened observation vector as input and processes it directly through fully connected layers to produce the policy output. This

model does not use permutation invariance, meaning that the ordering of the vehicles can affect the policy output. As a result, the MLP may struggle to generalise when similar traffic situations are represented with different vehicle orderings. It was included as the simplest baseline and used as a reference point for comparison with the other models.

3.3.2 Deep Sets with mean pooling

The Deep Sets with mean pooling architecture processes the observation as a set of vehicle vectors rather than treating it only as a single flattened input. Each vehicle is passed through a shared encoder network individually to produce a learned embedding. The vehicle embeddings are then aggregated using mean pooling to create a permutation-invariant traffic summary. As mean pooling is a symmetric operation, changing the ordering of the vehicles in the observation has no effect on the output.

3.3.3 Deep Sets with mean and max pooling

The Deep Sets architecture with mean and max pooling extends the mean pooling model by using two aggregation operations instead of one. After each observed vehicle is passed through the shared encoder to produce a learned embedding, the resulting set of embeddings is aggregated using both mean pooling and max pooling. These two pooled representations are then concatenated to form a more informative traffic summary. Mean pooling captures the overall traffic context, while max pooling helps preserve the strongest vehicle features, such as the neighbouring vehicle with the most immediate hazard. Since both operations are symmetric, changing the order does not affect the final output of the policy network.

3.3.4 Self-attention model

The self-attention architecture takes a different approach to tackling the vehicle-ordering problem by using attention to model the relationships and relative importance between vehicle embeddings when forming the traffic representation. This makes the model more expressive, but also more complex to train. In theory, this allows the model to preserve richer information about surrounding traffic than fixed pooling operations such as mean or max pooling.

3.4 Training and evaluation protocol

3.4.1 Training configuration

All model architectures were trained under the same PPO settings to ensure a fair comparison. Each model was trained for 100,000 timesteps, with 4096 timesteps per batch and an episode cap of 300 during training. Across all models, the learning rate was set to $1e-4$, the discount factor was set to 0.99, the PPO clipping parameter was 0.2, and the number of policy updates per iteration was fixed at 5. The action space remained the same for all models, and the covariance of the action distribution was fixed at 0.08.

3.4.2 Evaluation procedure

For evaluation, the mean action was used deterministically rather than sampled stochastically as in training. Each model was evaluated over 50 episodes, with each episode capped at 600 steps. For each episode, the episodic return and whether a collision occurred were recorded. The collision rate was then computed across all evaluation episodes.

3.5 Backbone selection results & analysis

3.5.1 Quantitative results

This section compares the four backbone architectures using average episodic return, average episode length and collision rate. Average return measures overall task performance, episode length indicates how long the agent survives before termination, and collision rate measures safety. These metrics were chosen to select the model that provides the best balance between performance and safety.

The graphs below show the evaluation results and final training iteration for all models considered for backbone selection. Each model was evaluated over 50 episodes under the same conditions.

Model	Avg episodic return	Avg episodic length	Collision rate
Basic MLP	150.75	187.75	0.70
Deep Sets (mean)	163.51	204.19	0.78
Deep Sets (mean+max)	181.96	225.75	0.73
Self-attention	136.31	170.13	0.86

Table 1: Full evaluation results for backbone selection for over 50 episodes

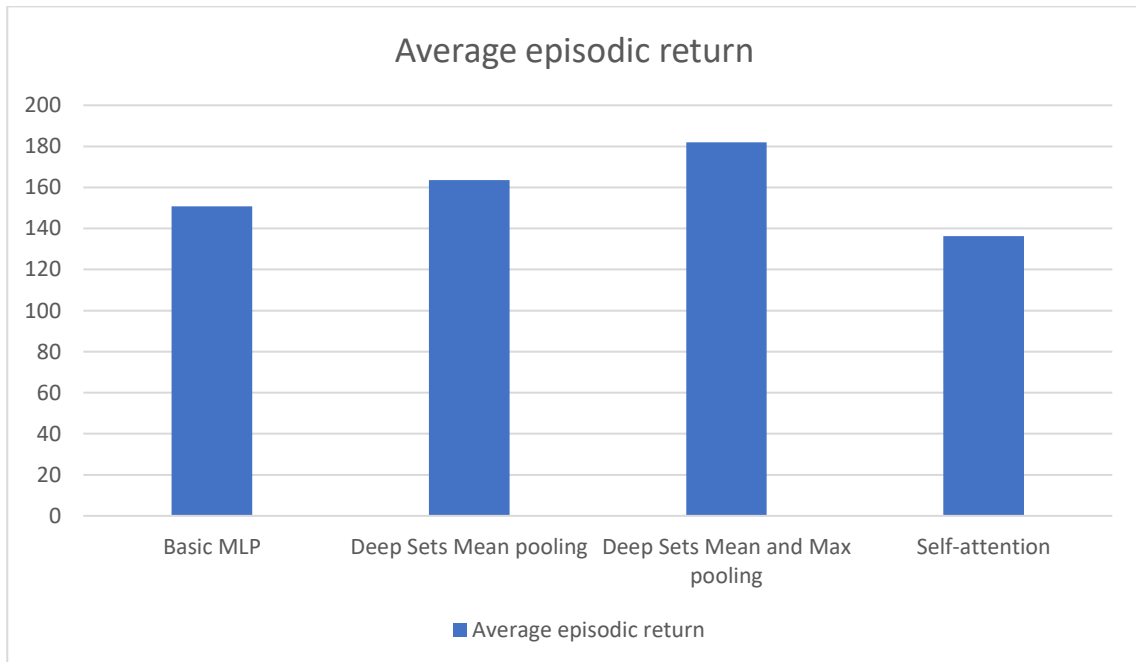


Figure 1: Average episodic return comparison between the four potential backbone architectures

Figure 1 shows that the Deep Sets mean+max backbone achieved the highest average episodic return, followed by Deep Sets mean, Basic MLP and self-attention. This

suggests that combining mean and max pooling produced the strongest reward-based performance.

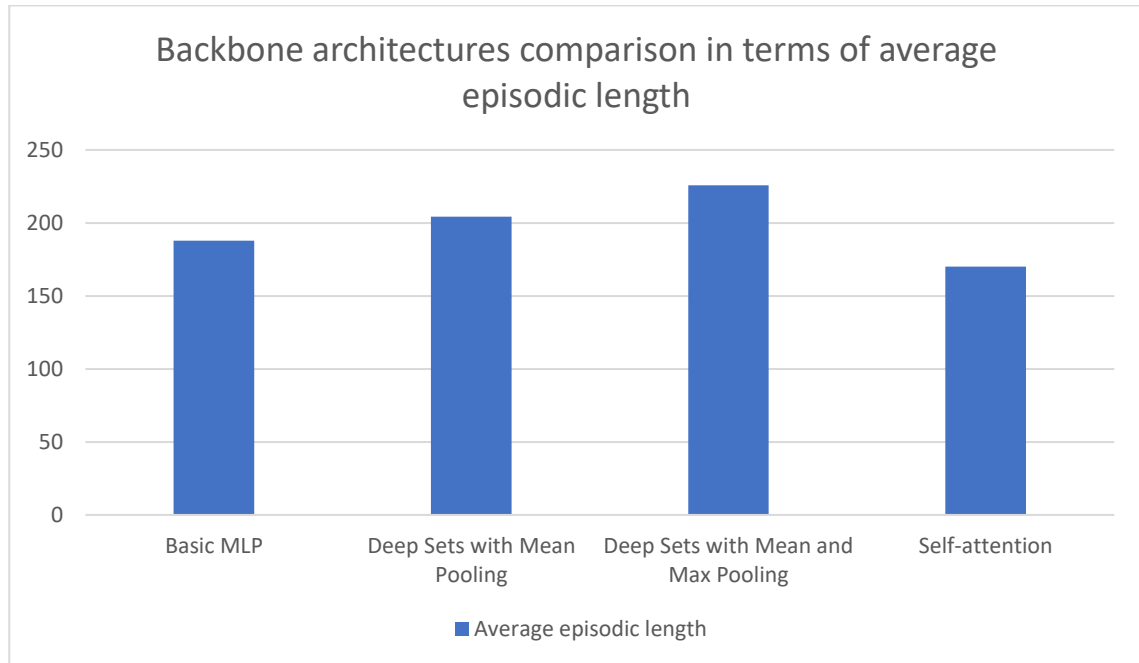


Figure 2: Average episodic length comparison between the four potential backbone architectures

Figure 2 shows that the Deep Sets mean+max backbone achieved the longest average episode length, followed by Deep Sets mean, Basic MLP and self-attention. This suggests that combining mean and max pooling helped the agent survive for longer.

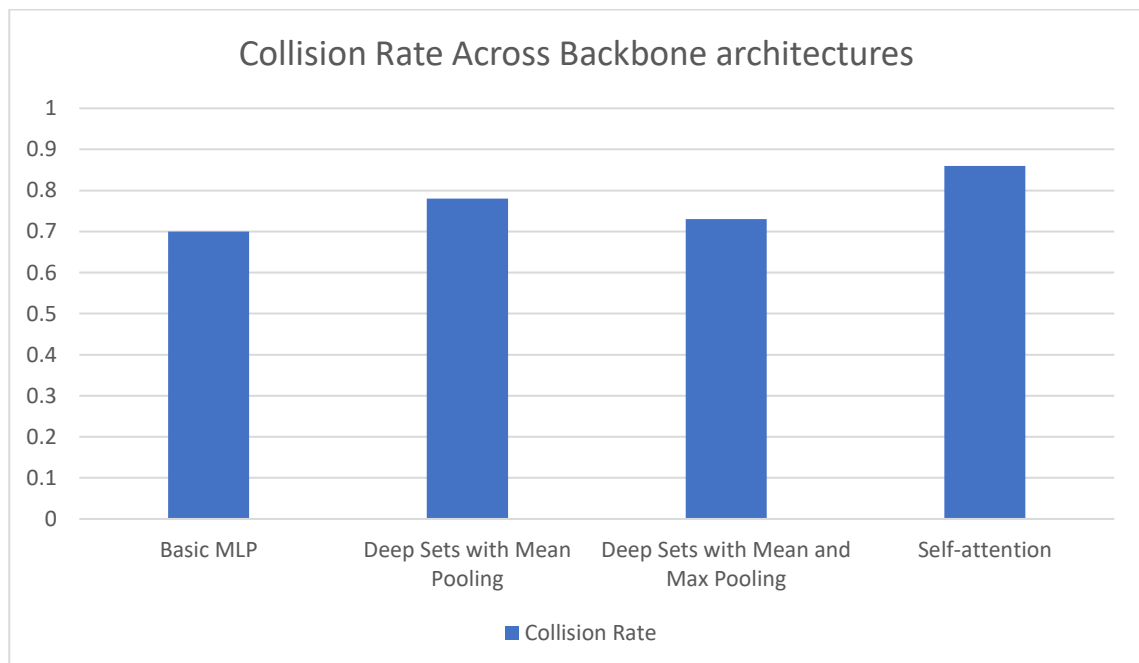


Figure 3: Average collision rate comparison between the four potential backbone architectures

Figure 3 shows that the Basic MLP model achieved the lowest collision rate, while self-attention had the highest collision rate. Although Deep Sets mean+max did not have the lowest collision rate, the difference between it and the Basic MLP was not significant.

Model	Final Training return	Final training length
Basic MLP	22.26	37.12
Deep Sets (mean)	25.25	36.04
Deep Sets (mean+max)	29.61	42.11
Self-attention	94.03	117.94

Table 2: Final training iteration results for backbone selection

The results show a trade-off between performance and safety. Deep Sets mean+max produced the strongest return and survival performance, while the Basic MLP achieved the lowest collision rate. This indicates that maximising return did not perfectly align with minimising collisions.

3.5.2 Training vs Evaluation performance

The results show that good performance in training does not always lead to good performance in evaluation. This is shown by the self-attention model, which achieved the highest final training return but performed worst during evaluation. This suggests that training return alone was not reliable for selecting the final backbone. Evaluation performance was therefore prioritised because it better reflects how the trained policy behaves in new settings.

3.5.3 Backbone performance analysis

The backbone comparison showed that the choice of policy architecture had a clear effect on the agent's performance. The Basic MLP achieved the lowest collision rate, which suggests that it produced the safest behaviour. However, it also achieved a lower average episodic return and shorter episodic length than both Deep Sets models, highlighting weaker overall driving performance.

The Deep Sets mean model performed better than the basic MLP in terms of average return and episode length. This suggests that treating nearby vehicles as a set, rather than a flattened vector, helped the agent form a more useful traffic representation. However, its collision rate was higher than the basic MLP, showing that improving return and episodic length does not automatically lead to safer driving behaviour.

The Deep Sets mean+max model achieved the strongest overall evaluation performance, with the highest average episodic return and longest average episodic length. This suggests that combining mean pooling and max pooling provided the agent with a more useful traffic representation consisting of the general traffic context and the

most important vehicle features. Although its collision rate was not the lowest, it provided the best balance between performance and safety among the tested architectures.

The self-attention model achieved the highest final training return, but this did not transfer to evaluation, where it had the weakest overall metrics. This suggests that the self-attention model may have been less stable under the current training budget or more sensitive to the evaluation configuration. This highlights that training performance alone was not sufficient for selecting the best backbone.

3.5.4 Backbone Selection

Based on the evaluation results, the Deep Sets mean+max architecture was selected as the backbone for the uncertainty-aware experiments. This model achieved the highest average episodic return and the longest average episode length, suggesting that it produced the strongest overall driving performance during inference. Although the Basic MLP achieved the lowest collision rate, its much lower return and episode length suggest that it is a weaker architecture compared with the mean+max architecture. Therefore, Deep Sets mean+max was chosen because it provided the best balance between reward, episodic length and safety.

3.6 Uncertainty-aware PPO formulation

3.6.1 Monte Carlo dropout critic

To compute uncertainty, Monte Carlo dropout was used in the critic network while leaving the actor network unchanged. This allowed the value function to produce an uncertainty estimate alongside the state-value prediction.

A dropout layer was used between layers in the critic network so that each forward pass sampled a slightly different sub-network. Repeated evaluations of the same state therefore produced slightly different value estimates, which were then used to approximate uncertainty from the spread of these predictions.

The following formula computes the mean value estimate:

$$\hat{V}(s) = \frac{1}{K} \sum_{k=1}^K V_k(s)$$

where K is the number of Monte Carlo samples and the mean is the estimated state value.

The uncertainty estimate is computed as the variance across the predicted value estimates:

$$u(s) = \text{Var}(V_1, V_2, \dots, V_k)$$

where $u(s)$ is the estimated uncertainty.

A higher variance indicates that the critic is less confident in its value prediction, whereas a lower variance indicates greater confidence.

In this project, a dropout probability of 0.1 and 5 Monte Carlo samples were used when estimating critic uncertainty. These values were chosen to provide a lightweight estimate

without significantly increasing computational cost. The produced uncertainty estimate was then incorporated into the PPO algorithm through an uncertainty penalty term, described in the next subsection.

3.6.2 Uncertainty penalised reward

To incorporate uncertainty into policy learning, the estimated uncertainty produced by the critic was used to alter the reward signal during training. The original reward was replaced with an uncertainty-penalised reward, as shown in the following formula:

$$r'_t = r_t - \lambda_u u_t$$

where the scaling factor controls the magnitude of the uncertainty penalty, and the uncertainty term is the estimated uncertainty. Normalisation was applied so that the uncertainty term remained on a comparable scale and did not overwhelm the original reward signal.

The purpose of this penalty was to discourage the agent from visiting highly uncertain states, which should encourage more predictable behaviour. In this project, the uncertainty penalty strength was treated as a hyperparameter, with different values explored to assess its effect on collision rate, episodic return and episodic length.

3.6.3 Experimental purpose

The following subsection compares the baseline PPO model, using Deep Sets with mean and max pooling as the backbone, with the PPO model using an uncertainty-penalised reward built on the same backbone. In this comparison, the environment, training budget and PPO settings were kept constant to ensure a fair comparison. The comparison focuses on whether uncertainty-aware PPO could improve average episodic return, episodic length and collision rate.

3.7 Baseline & Uncertainty model comparison

3.7.1 Quantitative Results

Model	λ_u	Average episodic return	Average episodic length	Collision rate
Deep Sets mean+max baseline	0.00	181.96	225.75	0.733
Uncertainty penalty PPO	0.01	218.05	267.18	0.68
Uncertainty penalty PPO	0.05	187.74	232.6	0.74

Table 3: Full evaluation results for the comparison between Baseline and uncertainty penalty model for over 50 episodes

The results show that the uncertainty penalty model with a value of 0.01 achieved a slightly higher average return than the baseline, but also had a slightly higher collision rate. Increasing the penalty value to 0.05 reduced both episodic return and safety performance, leading to the highest collision rate among the compared models.

3.7.2 Analysis of Results

Compared with the baseline, the uncertainty penalty model using a value of 0.01 achieved a slightly higher average return, but also produced a slightly higher collision rate. This suggests that the addition of a small uncertainty penalty did not lead to a clear safety improvement over the baseline.

When the uncertainty penalty value was increased to 0.05, the uncertainty penalty model performed worse across all metrics. This suggests that a stronger uncertainty penalty did not make the policy safer and instead degraded overall performance.

Overall, these results suggest that the uncertainty penalty influenced learning, but not in the intended way. One possible reason is that the critic uncertainty estimate was only used as a training-time reward penalty, which may not align closely enough with collision risk. As a result, penalising the agent for moving into uncertain states more heavily did not necessarily lead it toward safer behaviour.

3.7.3 Summary of findings

In summary, the results indicate that the uncertainty penalty extension did not lead to a clear improvement in safety. Although the uncertainty penalty model with a value of 0.01 had a slightly higher average return than the baseline, both uncertainty penalty models had higher collision rates. Therefore, under the current configurations, the uncertainty penalty extension did not result in safer driving behaviour.

3.8 Inference-time Uncertainty Safeguard

3.8.1 Motivation

The previous uncertainty penalty experiment only used uncertainty during training. This meant that uncertainty influenced the policy indirectly through the reward signal, but it did not directly affect action selection during evaluation. Therefore, an additional experiment was introduced to test whether critic uncertainty could be more useful as an inference-time safeguard.

3.8.2 Safeguard method

During evaluation, Monte Carlo dropout was kept active in the critic network. For each state, the critic was sampled multiple times, and the variance of the value estimates was used as an approximation of uncertainty. If the uncertainty exceeded a fixed threshold, the safeguard was activated.

When activated, the safeguard capped acceleration and reduced steering magnitude. The aim was to make the policy more conservative in uncertain states while leaving the original actor policy unchanged in lower uncertainty states.

3.8.3 Experimental Setup

The threshold model used the same Deep Sets mean+max backbone, environment configuration, 100,000 training timesteps, hyperparameters and fixed evaluation seeds

as the baseline comparison. Evaluation was deterministic and used 50 episodes per trained model.

3.8.4 Results

Model	Avg episodic return	Avg episodic length	Collision rate
Deep Sets mean+max baseline	181.96	225.75	0.733
Uncertainty threshold mean+max backbone	216.97	267.35	0.66

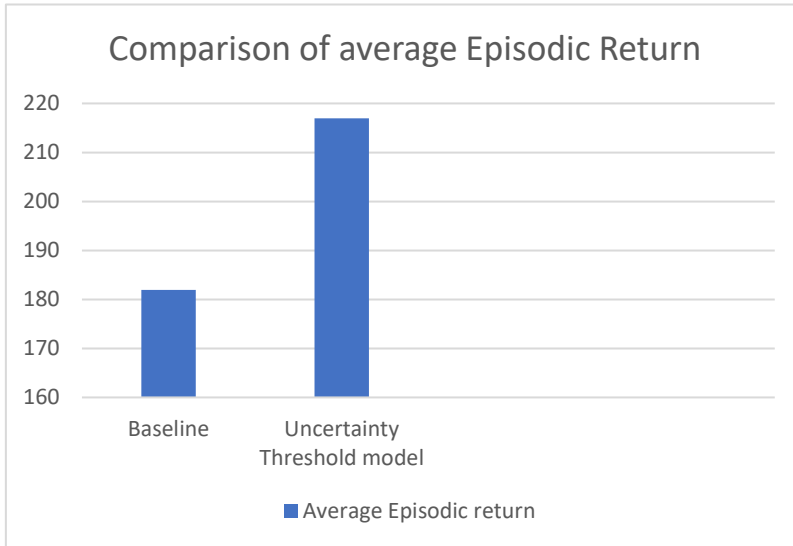


Figure X: Average episodic return comparison between the Deep Sets mean+max baseline and the inference-time uncertainty model

Figure X shows that the inference-time uncertainty threshold model achieved a higher average episodic return than the baseline. The threshold model achieved an average return of 216.97 across four runs, compared with 181.96 for the baseline across three runs. This indicates that the safeguard improved overall driving performance.

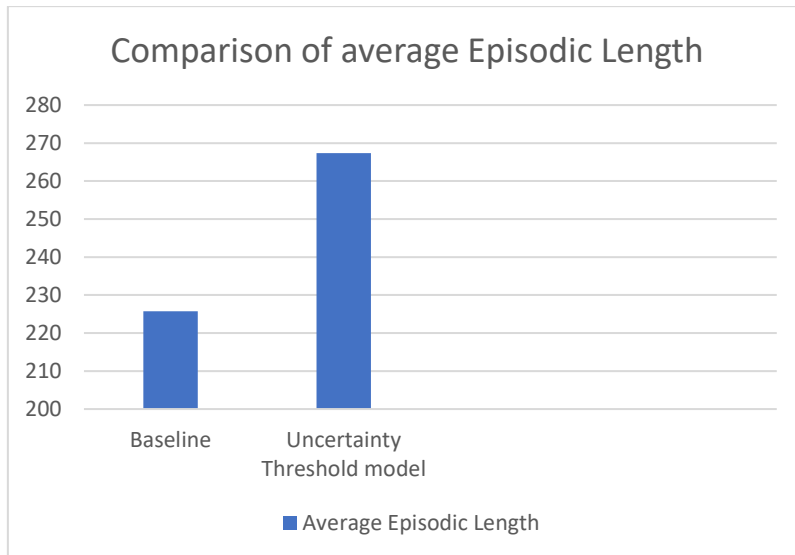


Figure X: Average episodic length comparison between the Deep Sets mean+max baseline and the inference-time uncertainty model

Figure X shows that the inference-time uncertainty threshold model achieved a higher average episodic length than the baseline. The threshold model achieved an average episodic length of 267.35 across four runs, compared with 225.75 for the baseline across three runs. This indicates that the safeguard improved overall survival time.

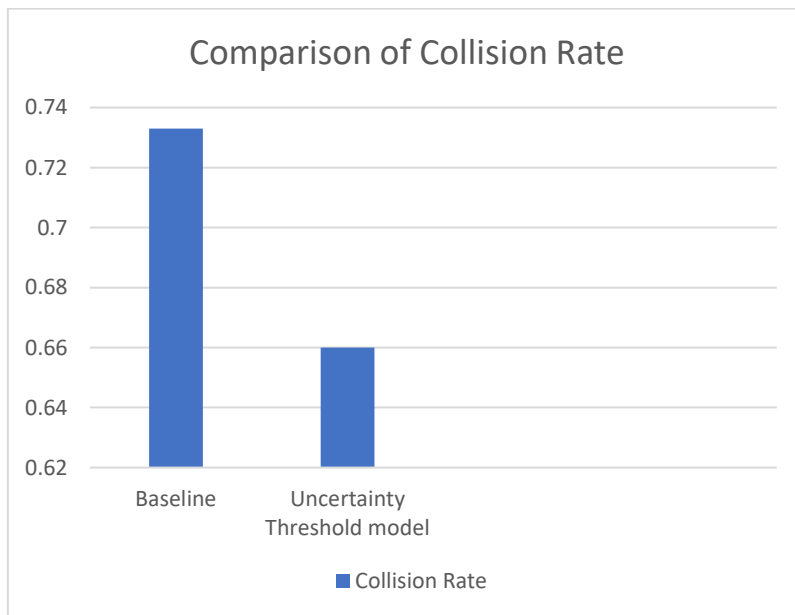


Figure X: Collision rate comparison between the Deep Sets mean+max baseline and the inference-time uncertainty model

Figure X shows that the inference-time uncertainty threshold model had a lower collision rate than the baseline. The threshold model had a collision rate of 0.66 across four runs, compared with 0.733 for the baseline across three runs. This indicates that the safeguard improved overall safety of the RL agent.

The graph above shows that the uncertainty threshold model has a lower collision rate of 0.66 compared with the baseline, which has a collision rate of 0.733.

3.8.5 Summary

The threshold-based uncertainty model achieved a higher average episodic return and a lower collision rate than the baseline. The average return increased from 181.96 to 211.04, while the collision rate decreased from 0.733 to 0.684. This suggests that using uncertainty directly during action selection may be more effective than using it only as a training-time reward penalty.

Overall, the training-time uncertainty penalty did not clearly improve safety. However, the later threshold-based safeguard produced an improvement in both average return and collision rate compared with the Deep Sets mean+max baseline. This suggests that critic uncertainty may be more useful when applied directly at inference time, although the results remain sensitive to the training seed and chosen threshold value.

3.9 Limitations

3.9.1 Training budget

A limitation of this project is that the training budget was fixed at 100,000 timesteps for all models. Although this ensured a fair comparison, it may not have been enough for more complex architectures, such as self-attention, which may require more interactions with the environment to reach their full potential. As a result, the self-attention model may not have fully converged within the allocated budget. Therefore, the self-attention architecture ranking below the Deep Sets architecture may be due to differences in learning speed rather than model quality.

3.9.2 Evaluation variance

A further limitation of the evaluation procedure is the presence of variance. Although the policy remained fixed during each of the 50 evaluation episodes, the environment was stochastic, with vehicles starting in different positions, lanes and traffic layouts. This led to the agent surviving for the full episode length in some episodes and crashing early in others. Therefore, the metrics produced are sample-based estimates, as they are computed from a finite set of randomly generated evaluation episodes rather than the full distribution of all possible traffic scenarios. These metrics are therefore approximations that may vary depending on the specific evaluation seeds used.

To reduce this limitation, a fixed set of evaluation seeds was used across all models, improving the fairness of the comparison.

3.9.3 Limitation of the uncertainty penalty

A key limitation of the uncertainty penalty model was that it was only used as a training reward penalty and was not used directly at inference time. As a result, it did not explicitly prevent uncertain actions during evaluation, but instead influenced behaviour indirectly by penalising the agent's rewards during training. Therefore, the uncertainty estimate influenced learning indirectly through PPO updates rather than directly affecting decision-time behaviour.

Another limitation is that the critic's uncertainty is not the same as action risk. This is because uncertainty was estimated through the critic's value prediction rather than from a direct crash-risk model. For example, the uncertainty estimate could be high because the state is rare or unexplored, without necessarily being dangerous. Similarly, the

uncertainty estimate could be low for a dangerous state if the critic is confident in its value estimate. Consequently, the penalty may push the agent away from uncertain value estimates, which does not always align with avoiding danger.

Another limitation is the strength of the penalty. When the penalty value is small, such as 0.01, the uncertainty term has very little influence on the agent's learning, resulting in behaviour that is similar to the baseline. When the penalty value is larger, such as 0.05, it may distort learning because the uncertainty term has much more influence, resulting in reduced performance without improving safety.

3.9.4 Limitation of threshold safeguard

The threshold safeguard used a fixed uncertainty threshold of 0.0005. This threshold was manually selected and may not be optimal for all traffic states or training seeds. Moreover, the critic uncertainty is not a direct measure of collision risk, so the safeguard may activate in states that are uncertain but not dangerous, or fail to activate in dangerous states where the variance is low. The safeguard action may also have been too limited because it only capped acceleration and dampened steering, so future work could explore more adaptive interventions.

3.10 Inference-time Uncertainty Safeguard

Chapter 4: Presentation

4.1 Source Code

Use the Code style for presenting code snippets within this report.

```
static public void main(String[] args) {  
    try {  
  
        UIManager.setLookAndFeel(UIManager.getSystemLookAndFeelClassName());  
    }  
    catch(Exception e) {  
        e.printStackTrace();  
    }  
    new WelcomeApp();  
}
```

Chapter 5: **Evaluation**

Chapter 6: Conclusion

6.1 Project Summary

This project investigated the use of PPO for autonomous driving in a simulated highway environment. The project was split into two stages. The first compared several policy network backbones in order to identify a suitable representation for processing traffic observations. The second extended the selected backbone with an uncertainty approximation using Monte Carlo dropout in the critic network. Together, these two stages were designed to examine both the effect of policy representation on driving performance and whether uncertainty estimation would lead to safer agent behaviour.

6.2 Summary of Key findings

The first stage of the project showed that the choice of policy backbone had a clear effect on driving performance. Out of all the architectures tested, the Deep Sets mean+max model achieved the strongest overall evaluation results, outperforming the Basic MLP, Deep Sets mean and self-attention architectures. Although the self-attention model achieved the highest final training return, this did not translate into the best overall metrics, indicating that greater model complexity did not necessarily lead to better generalisation under the fixed training budget.

The second stage evaluated whether extending the PPO algorithm with uncertainty approximation via Monte Carlo dropout would improve performance or safety relative to the baseline. Monte Carlo dropout was successfully added to the critic to estimate epistemic uncertainty, and the uncertainty was then used as a penalty term. However, the results did not show a significant improvement in performance for the models using the uncertainty penalty, and the collision rate actually increased. This indicates that, under the tested setup, uncertainty penalisation during training was not sufficient to improve safety or performance.

References

Clements, W.R., Van Delft, B., Robaglia, B.-M., Slaoui, R.B. and Toth, S. (2019) 'Estimating risk and uncertainty in deep reinforcement learning', arXiv:1905.09638. Available at: <https://arxiv.org/abs/1905.09638> (Accessed: 11 May 2026).

Farama Foundation (n.d.) 'highway-env documentation'. Available at: <https://highway-env.farama.org/> (Accessed: 11 May 2026).

Gal, Y. and Ghahramani, Z. (2016) 'Dropout as a Bayesian approximation: Representing model uncertainty in deep learning', Proceedings of the 33rd International Conference on Machine Learning, 48, pp. 1050–1059. Available at: <https://proceedings.mlr.press/v48/gal16.html> (Accessed: 11 May 2026).

Ghasemi, M. and Ebrahimi, D. (2024) 'Introduction to reinforcement learning', arXiv:2408.07712. Available at: <https://arxiv.org/abs/2408.07712> (Accessed: 11 May 2026).

Gu, S., Grudzien Kuba, J., Chen, Y., Du, Y., Yang, L., Knoll, A. and Yang, Y. (2023) 'Safe multi-agent reinforcement learning for multi-robot control', Artificial Intelligence, 319, Article 103905. Available at: <https://doi.org/10.1016/j.artint.2023.103905> (Accessed: 11 May 2026).

Kendall, A. and Gal, Y. (2017) 'What uncertainties do we need in Bayesian deep learning for computer vision?', Advances in Neural Information Processing Systems, 30. Available at: <https://papers.nips.cc/paper/7141-what-uncertainties-do-we-need> (Accessed: 11 May 2026).

Leurent, E. and Mercat, J. (2019) 'Social attention for autonomous decision-making in dense traffic', NeurIPS 2019 Machine Learning for Autonomous Driving Workshop. Available at: <https://arxiv.org/abs/1911.12250> (Accessed: 11 May 2026).

Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017) 'Proximal policy optimization algorithms', arXiv:1707.06347. Available at: <https://arxiv.org/abs/1707.06347> (Accessed: 11 May 2026).

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention is all you need', Advances in Neural Information

Processing Systems, 30. Available at: <https://arxiv.org/abs/1706.03762> (Accessed: 11 May 2026).

Zaheer, M., Kottur, S., Ravanbakhsh, S., Poczos, B., Salakhutdinov, R. and Smola, A. (2017) 'Deep Sets', Advances in Neural Information Processing Systems, 30. Available at: <https://arxiv.org/abs/1703.06114> (Accessed: 11 May 2026).

Zhao, W., He, T. and Liu, C. (2023) 'Probabilistic safeguard for reinforcement learning using safety index guided Gaussian process models', Proceedings of Machine Learning Research, 211, pp. 1–33. Available at: <https://proceedings.mlr.press/v211/zhao23a/zhao23a.pdf> (Accessed: 11 May 2026).

Appendix A – Example

Model	Avg episodic return	Avg episodic length	Collision rate
Basic MLP	150.75	187.75	0.70
Deep Sets (mean)	163.51	204.19	0.78
Deep Sets (mean+max)	181.96	225.75	0.73
Self-attention	136.31	170.13	0.86

Table 1: Full evaluation results for backbone selection for over 50 episodes

Additional Appendices (as needed)